Learning Visibly Pushdown Languages under Rewriting Rules

Jan Otop
University of Wrocław

Piotr Traczyński
University of Wrocław

Papers must not exceed 15 pages (excluding references and appendices, LIPIcs style)

Some lemmas to theorems

Abstract

We present a method for approximating Visibly Pushdown Languages (VPLs) using string rewriting rules. We apply this framework to extend Angluin-style active learning for VPLs by incorporating deductive inference from rewrite rules. String rewriting rules that encode prior knowledge about the target language and the learning algorithm can deductively infer answers to some queries without asking the oracle, thereby reducing its query complexity. We demonstrate the efficacy of this approach through examples of rewrite systems that capture natural properties of structured data and significantly reduce the number of equivalence queries directed to the oracle. Finally, we address the rule synthesis problem, which is a first step for making active automata learning algorithms explainable.

2012 ACM Subject Classification Theory of computation → Grammars and context-free languages; Theory of computation → Rewrite systems

Keywords and phrases visibly pushdown automata, string rewriting, active learning

Digital Object Identifier 10.4230/LIPIcs...

1 Introduction

Active automata learning. The aim of *active automata learning* is to generate a *deterministic finite automaton (**DFA**)* recognizing an unknown regular language $\mathcal{L}$ (target language) given access to an oracle that answers queries about $\mathcal{L}$. Unlike *passive* learning, where the input dataset is fixed in advance, the algorithm may *actively* query the language. There are two types of queries: *membership* of a word in $\mathcal{L}$, and *equivalence* of the language of a given **DFA** $\mathcal{A}$ with the target language $\mathcal{L}$, where the oracle either confirms equivalence or returns a *counterexample* word witnessing the difference between $\mathcal{L}$ and $\mathcal{L}(\mathcal{A})$. This framework was proposed in the seminal paper [6] together with the L*-algorithm, which enables learning in polynomial time. Active automata learning has been intensively researched for both efficiency [31, 52] as well as extensions to other automata-based models such as nondeterministic finite automata [16], tree automata [23], nominal automata over infinite alphabets [46] and weighted automata [11].

Learning context-free languages. The active learning framework has been extended to pushdown languages [35, 7, 21, 10, 39], where the representations typically considered have been context-free grammars However, generating context-free grammars from queries is substantially more difficult than generating **DFA**, as there is no counterpart to the Myhill-Nerode theorem for pushdown languages. Furthermore, the equivalence problem for context-free grammars is undecidable, and for deterministic context-free grammars, which are equivalent to deterministic pushdown automata, the best currently known upper bound on the complexity of deciding equivalence is primitive recursive [48, 32]. However, a

© Jan Otop and Piotr Traczyński;
licensed under Creative Commons License CC-BY 4.0
Leibniz International Proceedings in Informatics
LIPICS Schloss Dagstuhl – Leibniz-Zentrum für Informatik, Dagstuhl Publishing, Germany

subclass of context-free languages called *visibly pushdown languages (**VPLs**)* [43, 3], where the stack operations are determined by the input letters, admits a Myhill-Nerode-style characterization [2] and language equivalence of deterministic visibly pushdown automata (**DVPA**) is decidable in polynomial time [3].

Learning VPLs. While **DVPA** share multiple good properties with **DFA** such as polynomial-time closure over Boolean operations and subset-based determinization, the minimization problem for **DVPA** is NP-complete [28], which rules out polynomial-time active learning algorithms for **DVPA** (unless ¶=NP) There are different approaches to address this problem including restricting the learning problem to a subclasses of **DVPA** [40], consider a different automata model for language representation, called k-SEVPA [30, 18, 34], which may involve exponential blow-up upon transformation to **DVPA** [2], or augmenting the oracle with queries that depend on the structure of the target **DVPA** [44].

Applications of learning VPLs. Learning **VPLs** has been applied to a wide range of domains, including inferring specifications for document validation from JSON schemas, generation of valid program inputs [10, 34], and verification and explainability of Recurrent Neural Networks (RNNs) [9]. In these applications, equivalence queries are typically approximated because exact equivalence checking is often infeasible or undecidable. For instance, verifying the equivalence of a **DVPA** against a general program is undecidable, while comparison with an RNN inevitably yields a negative result, as standard RNNs are limited to recognizing regular languages. Furthermore, the non-equivalence witness can be prohibitively large [29], making the negative answer meaningless. These factors make answering equivalence queries a significant challenge in such settings.

Query complexity. Due to high cost of answering queries, there is a large body of work on reducing the query complexity of active automata learning [31, 27, 52, 38, 22] as well as implementing the teacher using Large Language Models [42, 14, 25]. However, LLMs can only answer membership queries, and equivalence query answers are merely approximated by a large number of membership queries, which is also the standard approach to equivalence queries in other settings [51]. Consequently, we adopt the alternative approach proposed in [26], which restricts the algorithm's search space by supplying an additional information regarding the target language called *advice*. Specifically, we focus on inferring responses to equivalence queries, as this yields the greatest reduction in query complexity and improves the overall reliability of the learning process, particularly in contexts where equivalence queries are typically approximated.

Learning with advice. We consider an active learning framework augmented with advice, where the learning algorithm is provided with *advice* expressing prior knowledge regarding the target language [26]. By using the advice to deductively infer query answers, this framework bridges two synthesis paradigms: deductive synthesis from specifications and inductive synthesis via queries. This hybrid approach allows a language to be represented partially through queries and partially through advice expressed as a *string rewriting system (SRS)*. Since SRSs and automata operate on fundamentally different principles, certain language properties admit far more succinct descriptions as rewriting rules than through query-answer interactions.

String rewriting rules for pushdown languages. A String Rewriting System (SRS) consists of rewrite rules $l \to r$, where l, r are words. *Rewriting* is the process of iteratively transforming words by replacing the infix that matches the left-hand side of a rule $\boldsymbol{\alpha} \to \boldsymbol{\beta}$ with its corresponding right-hand side $\boldsymbol{\beta}$. For instance, commutativity of letters a, b can be expressed with $ab \to ba$. However, if a is a call letter and b is a return letter, the commutativity rule disrupts the matching between call and return letters. We discuss

this issue in the paper and define a class of rules, which are compatible with the implicit structure of pushdown languages. Furthermore, in the pushdown case, there are properties not expressible by simple rewrite rules such as all content between comment tags can be discarded. We address this issue by extending rewrite rules with variables.

Modelling prior knowledge with string rewriting. In our framework, an advice SRS induces an equivalence relation on words relative to the target language $\mathcal{L}$. Specifically, if a word w belongs to $\mathcal{L}$, then every word obtained by rewriting w must also be in $\mathcal{L}$; symmetrically, rewriting must respect the complement of $\mathcal{L}$. We also consider one-sided variants based on implication stating that (1) words from the language may only be rewritten words in the language, while there are no restrictions for words outside the language, or (2) words outside the language may only be rewritten words outside the language. This allows the SRS to capture high-level structural properties, such as commutativity or idempotence, without necessarily characterizing the learned language completely.

Contributions. While active automata learning with advice was introduced in [26], extending this framework from regular languages to **VPLs** and Deterministic Visibly Pushdown Automata (DVPAs) is non-trivial. This transition requires addressing structural challenges of pushdown systems, which are absent in the regular case. The main contributions of presented in this paper are:

1. We identify string rewriting rules that are compatible with the structural properties of pushdown languages. Furthermore, we extend these rules from word pairs to rules with variables in order to capture basic pushdown language properties.
2. We develop an algorithm that infers answers to equivalence queries for **DVPA**.
3. We establish the complexity of the rule synthesis problem.
4. We demonstrate the practical benefit of the advice mechanism through experiments

The detailed proofs of presented results are relegated to the appendix.

Related work. There is a large body of work on active learning of context-free grammars [35, 50, 5, 47, 49, 21, 10, 12, 39, 55], as well as in learning visibly pushdown grammars and visibly pushdown automata [30, 44, 18, 9, 34, 24]. These works primarily aim to improve learning algorithms for arbitrary input languages by exploring alternative representation models, augmenting the learning framework with additional capabilities (e.g., oracles that answer additional types of queries, or assuming the set of non-terminals is known), or altering the mode of learning Improving the performance of active learning algorithms by restricting learned languages to particular automaton subclasses is an active research direction for Mealy machines [1, 13, 41, 36, 45] and tree automata [53, 15]. In contrast to these works, we do not commit to any fixed subclass of **VPLs**; instead, the search space of our algorithm is parameterized by the advice SRS provided in the input.

2 Preliminaries

Pushdown alphabet. A pushdown alphabet $\widehat{\Sigma}$ consists of three disjoint sets of letters $\widehat{\Sigma} = (\Sigma_c, \Sigma_l, \Sigma_r)$ called, respectively, *call*, *local* and *return* letters.

Well-matched words. The set of well-matched words over a pushwdown alphabet $\widehat{\Sigma}$ is the least set containing ϵ and closed under concatenation such that (1) if w is well matched, c is a call letter and r is a return letter, then cwr is well matched, and (2) if w is well matched and l is a local letter, then lw and wl are well-matched. In a word w (not necessarily well matched), if $w = xcyrz$, the word y is well-matched, we say that letters c and r are matched, and r is the return letter corresponding to call c and vice versa. Formally, positions $|x| + 1$ and $|x| + |y| + 2$ are in the *matching relation*.

Visibly pushdown automata. A *(non-deterministic) visibly pushdown automaton* (**VPA**) $\mathcal{A}$ is a tuple $\mathcal{A} = (\widehat{\Sigma}, \Gamma, Q, Q_0, \delta, F)$ consisting of: (1) $\widehat{\Sigma}$, a pushdown alphabet, (2) Γ, a finite stack alphabet, (3) Q, a finite set of states, (4) $Q_0 \subseteq Q$, a set of initial states, (5) δ, a finite transition relation defined as below, and (6) $F \subseteq Q$, a set of accepting states. The transition relation δ is the union of relations $\delta_c \cup \delta_l \cup \delta_r$, where $\delta_c \subset Q \times \Sigma_c \times Q \times \Gamma$ $\delta_l \subset Q \times \Sigma_l \times Q$, and $\delta_r \subset Q \times \Sigma_r \times \Gamma \cup \{\bot\} \times Q$.

Deterministic visibly pushdown automata. A **VPA** $(\widehat{\Sigma}, \Gamma, Q, Q_0, \delta, F)$ is deterministic (**DVPA**) if $|Q_0| = 1$ and δ is a function being the union of three functions: $\delta_c \colon Q \times \Sigma_c \to Q \times \Gamma$, $\delta_l \colon \times \Sigma_l \to Q$ $\delta_r \colon \times \Sigma_r \times \Gamma \cup \{\bot\} \to Q$.

Semantics of VPA. Consider a **VPA** $\mathcal{A} = (\widehat{\Sigma}, \Gamma, Q, Q_0, \delta, F)$. A *configuration* of $\mathcal{A}$ is a pair consisting of the current state $q \in Q$ of $\mathcal{A}$ and its current stack content $u \in (\bot \cdot \Gamma^*)$. A **VPA** defines an infinite-state transition system over its configurations $Q \times (\bot \cdot \Gamma^*)$, where configurations $Q_0 \times \{\bot\}$ are initial. For $a \in \widehat{\Sigma}$, let $\to^a_{\mathcal{A}}$ be an auxiliary relation defined such that

- $(q, u) \to^a_{\mathcal{A}} (q', uB)$, if a is a call letter and $(q, a, q', B) \in \delta_c$,
- $(q, u) \to^a_{\mathcal{A}} (q', u)$, if a is a local letter and $(q, a, q') \in \delta_l$,
- $(q, uB) \to^a_{\mathcal{A}} (q', u)$, if a is a return letter $(q, a, B, q') \in \delta_r$ and $B \in \Gamma$, and
- $(q, \bot) \to^a_{\mathcal{A}} (q', \bot)$, if a is a return letter $(q, a, \bot, q') \in \delta_r$.

For a word $w = a_1 \dots a_n$, we define $\to^w_{\mathcal{A}}$ as the composition of relation $\to^{a_1}_{\mathcal{A}} \dots \to^{a_n}_{\mathcal{A}}$, i.e., $(q, u_0) \to^w_{\mathcal{A}} (q', u_n)$ if there exist intermediate configurations $(q_1, u_1), \dots, (q_{n-1}, u_{n-1})$ such that for each $1 \leq i \leq n$ we have $(q_{i-1}, u_{i-1}) \to^{a_i}_{\mathcal{A}} (q_i, u_i)$.

A word w is *accepted* by $\mathcal{A}$ if there are $q_0 \in Q_0$, $q \in F$ and $u \in \bot \cdot \Gamma^*$ such that $(q_0, \bot) \to^w_{\mathcal{A}} (q, u)$. The *language recognized by* $\mathcal{A}$, denoted $\mathcal{L}(\mathcal{A})$, is the set of words accepted by $\mathcal{A}$.

String rewriting. A *string rewriting system (SRS)* $\mathcal{R}$ over an alphabet Σ is a (possibly infinite) set of pairs of words (l, r) over Σ. A pair of words (l, r) from $\mathcal{R}$ is called a *string rewrite rule* and denoted with $l \to r$. For an SRS $\mathcal{R}$, we define a *single-step rewrite relation* $\to_{\mathcal{R}}$ over words from Σ^* as follows: for all $s, t \in \Sigma^*$, we have $s \to_{\mathcal{R}} t$ if and only if there are words $x, y \in \Sigma^*$ and a rewrite rule $(l, r) \in \mathcal{R}$ such that $s = xly$ and $t = xry$. The (string) rewriting relation $\to^*_{\mathcal{R}}$ is the transitive and reflexive closure of $\to_{\mathcal{R}}$. We will omit the $\mathcal{R}$ subscript if the SRS is clear from the context. For further details and notions on the topic of string rewriting see [17].

Active learning of languages and automata. The framework of active automata learning assumes an oracle, called a *Minimally Adequate Teacher*, which answers two types of queries about the target language $\mathcal{L}$:

- **membership queries**: given a $w \in \Sigma^*$, is $w \in \mathcal{L}$?, and
- **equivalence queries**: given an automaton $\mathcal{A}$, is $\mathcal{L}(\mathcal{A}) = \mathcal{L}$? If not, the teacher returns a counterexample, which is a word from the symmetric difference of $\mathcal{L}(\mathcal{A})$ and $\mathcal{L}$.

Additional types of queries are also possible [44]. The goal of the active learning algorithm is to return a minimal-size automaton recognizing the target language. The automaton model is connected to the type of the learned language: regular languages are represented by **DFA**, while **VPLs** are represented by **DVPA**, Single-Entry Visibly Pushdown Automata (SEVPA) or other restrictions of **DVPA** [2, 20]. Our method focuses on inferring answers to equivalence queries; hence, it does not depend on the specific active learning algorithm used, as long as it asks such queries.

3 Rewriting strings in visibly pushdown languages

We adapt an advice mechanism introduced for **DFA** in [26] to **DVPA**. The main aim of advice is to provide the learning with prior knowledge about the learned language, which is used to reduce the number of equivalence queries.

While extending the framework from regular to visibly pushdown languages is straightforward, both frameworks differ in considered rewriting rules. We characterize which rewrite rules are compatible with visibly pushdown languages and analyze the computational complexity of resolving queries using this advice.

3.1 Connecting SRS with VPL

The advice for the learning algorithm is given via string rewriting systems (SRSs), which are related to visibly pushdown languages through the notions of consistency:

▶ **Definition 1** ([26]). *Let $\widehat{\Sigma}$ be a pushdown alphabet, $\mathcal{R}$ be an SRS over $\mathcal{F}$ and $\mathcal{L}$ be a visibly pushdown language over $\widehat{\Sigma}$. We say that*

- *$\mathcal{R}$ is* (fully) consistent *with $\mathcal{L}$ if and only if for all words x, y, if $x \to_{\mathcal{R}}^{*} y$, then $x \in \mathcal{L} \iff y \in \mathcal{L}$.*
- *$\mathcal{R}$ is* positively consistent *with $\mathcal{L}$ if and only if for all words x, y, if $x \to_{\mathcal{R}}^{*} y$, then $x \in \mathcal{L} \implies y \in \mathcal{L}$.*
- *$\mathcal{R}$ is* negatively consistent *with $\mathcal{L}$ if and only if for all words x, y, if $x \to_{\mathcal{R}}^{*} y$, then $x \notin \mathcal{L} \implies y \notin \mathcal{L}$.*

Note that an SRS $\mathcal{R}$ is fully consistent with $\mathcal{L}$ if and only if it is both positively and negatively consistent.

Having the notions of consistency present the framework of active learning with advice.

Learning with advice. The active **DVPA** learning with advice problem is as follows. Given an *oracle* $\mathcal{O}$ answering membership and equivalence queries about the target visibly pushdown language $\mathcal{L}$ (and possibly other queries), and three *advice* SRSs $\mathcal{R}_{=}$, which is consistent with $\mathcal{U}$, and $\mathcal{R}_{+}$ (resp. $\mathcal{R}_{-}$), which is positively (resp., negatively) consistent with $\mathcal{U}$, the task is to construct a **DVPA** recognizing $\mathcal{L}$. The advice is used to approximate the equivalence in order to reduce the number of equivalence queries. Therefore, it can be integrated with any framework for active learning of visibly pushdown languages [9, 34] or **DVPA** [44] as long as the oracle answers membership and equivalence queries. In particular, the oracle can answer other types of queries as in [44].

Reducing the query complexity regarding equivalence oracles. We can reduce the number of equivalence queries without affecting the learning algorithm. When the learning algorithm attempts to asks the oracle an equivalence query, we can capture the query and first check whether an advice SRS $\mathcal{R}$ is consistent (resp., positively consistent or negatively consistent) with the language of a given candidate automaton $\mathcal{A}$. If it is inconsistent, we know that $\mathcal{L}(\mathcal{A})$ differs from the target language, with which $\mathcal{R}$ is consistent (resp., positively consistent or negatively consistent). It suffices to compute a pair of words violating consistency (resp., positively consistent or negatively consistent), i.e., words x, y such that $x \to_{\mathcal{R}}^{*} y$ but $x \in \mathcal{L} \implies y \in \mathcal{L}$ does not hold. Then, one of the words x, y has to be a counterexample to $\mathcal{L}(\mathcal{A})$ being equal to the target language. It can be decided which one with a single membership query.

3.2 Rewrite rules for visibly pushdown languages

While the notion of consistency with the language is the same for **DVPA** as for **DFA**, the main difference lies in the rewrite rules.

In the case of regular languages, any pair of words is a valid string rewrite rule, but words in visibly pushdown languages have an additional implicit structure (which has been made explicit in [4]). The intuition behind rewrite rules is that words are rewritten to other words that are equivalent w.r.t. a given language. Therefore, rewrite rules should preserve the matching of pushdown letters except for letters in subwords being rewritten. Intuitively, rewriting one part of the word, should not affect other parts of the word, which is trivially true for words, but not for pushdown words. For instance, by rewriting $c_1c_2c_3r_3r_2r_1$ with the rule $c_3r_3 \to \epsilon r_3c_3$ we get a well-matched word with a very different matching $c_1c_2r_2c_3r_2r_1$.

▶ **Definition 2.** *A rewrite rule $\boldsymbol{\alpha} \to \boldsymbol{\beta}$* preserves matching *over $\widehat{\Sigma}$ if and only if for every word $w \in \widehat{\Sigma}^*]$ such that $w = x\boldsymbol{\alpha}y$ for some x, y we have: for every call letter c from x in $\boldsymbol{\alpha}$ has the corresponding return in y if and only if the same letter x in $w' = x\boldsymbol{\beta}y$ has the corresponding return letter in y.*

Matching preserving rules does not brake matching between calls and returns that are not in the rewritten fragment of the word. A prominent example of matching preserving rules are *well-matched rules*, which are natural and enjoy more efficient algorithms for checking consistency:

▶ **Definition 3.** *A rewrite rule $\boldsymbol{\alpha} \to \boldsymbol{\beta}$ is well-matched over $\widehat{\Sigma}$ if and only if words $\boldsymbol{\alpha}$ and $\boldsymbol{\beta}$ are well-matched.*

Clearly, every well-matched rule preserves matching. However, a rewrite rule $c_1c_2r_2 \to c_1$ is a matching preserving rule, while it is not a well-matched rule. Still, most of our examples are well-matched rules.

While we have restricted the set of admissible rules set to affect pushdown words locally, we must also extend it to capture the inherent properties of pushdown languages. We demonstrate this in the example below.

▶ **Example 4.** Consider the Dyck language $\mathcal{L}_2$ of well-matched brackets over tow pairs of brackets: $\{\langle, \rangle\}$ and $\{[,]\}$. The language $\mathcal{L}_2$ can be approximated by the property that the order of $\langle$ and $[$ is irrelevant as long as each opening bracket matches the closing one. For instance, in every word $\langle\,[\,]\,\rangle \to [\,\langle\,\rangle\,]$. Unfortunately, this property cannot be expressed by a finite set of string rewriting rules as it requires far apart letters to switch; for example $\langle\,[^n\langle\,\rangle\,]^n\rangle$ is equivalent to $\langle\,\langle\,[^n]^n\rangle\,\rangle$. However, we can express this property with a word variable X, for which any well-matched word can be substituted. Then, the rule

$$\langle\,[X]\,\rangle \to [\,\langle X\rangle\,]$$

expresses that the relative order of $\langle$ and $[$ is irrelevant as long as the corresponding brackets match.

Is consistency defined for rules with variables?

String rewriting systems with variables. Consider an infinite set of variables $\mathcal{X}$. A string rewriting system with variables (**SRS-V**) $\mathcal{R}[\mathcal{X}]$ is a finite set of pairs of words $(\boldsymbol{\alpha}, \boldsymbol{\beta})$ over $\widehat{\Sigma} \cup \mathcal{X}$ such that for every rule $\boldsymbol{\alpha} \to \boldsymbol{\beta} \in \mathcal{R}[\mathcal{X}]$, (1) in each $\boldsymbol{\alpha}$ and $\boldsymbol{\beta}$, every variable $X \in \mathcal{X}$ occurs at most once, and (2) if a variable $X \in \mathcal{X}$ occurs in $\boldsymbol{\beta}$, then it occurs in $\boldsymbol{\alpha}$ as well. We consider **SRS-Vs** as finite representations of infinite SRSs. That is, we say that a rule

$\boldsymbol{\alpha} \to \boldsymbol{\beta} \in \mathcal{R}[\mathcal{X}]$ generates an infinite SRS $[\![\boldsymbol{\alpha} \to \boldsymbol{\beta}]\!]$ obtained by all instantiations of variables from $\boldsymbol{\alpha}$ and $\boldsymbol{\beta}$ with well-matched words, where variable in both $\boldsymbol{\alpha}$ and $\boldsymbol{\beta}$ are instantiated to the same word. E.g. a rule $cXr \to X$ generates an infinite set $\{cr \to \epsilon, ccrr \to \epsilon cr, \ldots\}$. Finally, an **SRS-V** $\mathcal{R}[\mathcal{X}]$ generates an SRS $[\![\mathcal{R}[\mathcal{X}]]\!]$ defined as the union of $[\![\boldsymbol{\alpha} \to \boldsymbol{\beta}]\!]$ over all rules $\boldsymbol{\alpha} \to \boldsymbol{\beta}$ from $\mathcal{R}[\mathcal{X}]$.

Matching preserving and well-matched rules with variables. Consider a rewrite rule with variables $\boldsymbol{\alpha} \to \boldsymbol{\beta}$. We say that $\boldsymbol{\alpha} \to \boldsymbol{\beta}$ preserves matching (resp. is well-matched) if and only if the rule resulting from deletion of all variables from $\boldsymbol{\alpha} \to \boldsymbol{\beta}$ preserves matching (resp. is well-matched). Observe that if $\boldsymbol{\alpha} \to \boldsymbol{\beta}$ preserves matching (resp. is well-matched), then every rewrite rule for the generated SRS $[\![\boldsymbol{\alpha} \to \boldsymbol{\beta}]\!]$ preserves matching (resp. is well-matched).

We only assign well-matched words to variables as otherwise the resulting rules would not preserve matching. Consider a rule $cXr \to X$, which is well-matched if it is instantiated with a well-matched word for X. However, instantiating X with rc results in a rule $crcr \to rc$, which does not even preserve matching as $crcr$ is well-matched, while rc is not. Furthermore, considered only well-matched instantiations facilitates deciding consistency with a given **DVPA**.

3.3 Deciding consistency

We discuss the complexity of deciding consistency (resp., positive or negative consistency) of an **SRS** or **SRS-V** with the language of a given **DVPA** $\mathcal{A}$. Observe that an **SRS-V** $\mathcal{R}$ is consistent with the language of $\mathcal{A}$ if and only if for every rule $\boldsymbol{\alpha} \to \boldsymbol{\beta}$ and all words x, y we have $x\boldsymbol{\alpha}y \in \mathcal{L}(\mathcal{A}) \iff x\boldsymbol{\beta}y \in \mathcal{L}(\mathcal{A})$. Indeed, if this property holds for every rule, then by transitivity of the equivalence if $w \to^*_{\mathcal{R}} w'$, then $w \in \mathcal{L} \implies w' \in \mathcal{L}$. Similarly, it suffices to check consistency of single-step rewriting for positive and negative consistency.

Ground well-matched rules.. Consider a well-matched rule $\boldsymbol{\alpha} \to \boldsymbol{\beta}$ without variables, i.e., a standard string rewriting rule. We extend the alphabet $\widehat{\Sigma}$ with a fresh local letter # and construct two **DVPA** over the extended alphabet. Both **DVPA** extend $\mathcal{A}$ and differ only in behavior on the letter #; in $\mathcal{A}_1$ for any state q, the transition over # leads to the state q' such that $\mathcal{A}$ moves from q to q' over $\boldsymbol{\alpha}$. Since $\boldsymbol{\alpha}$ is well-matched, the run over $\boldsymbol{\alpha}$ does not depend on the stack content and it does not modify the stack content. Therefore, the **DVPA** $\mathcal{A}$ acts of $\boldsymbol{\alpha}$ as on a local letter. Next, the **DVPA** $\mathcal{A}_2$ is obtained in a similar way, but # acts as the word $\boldsymbol{\beta}$, which is also well-matched. Therefore, to check consistency of $\boldsymbol{\alpha} \to \boldsymbol{\beta}$ with $\mathcal{L}(\mathcal{A})$ it suffices to check whether there is a word w such that exactly one of **DVPA** $\mathcal{A}_1, \mathcal{A}_2$ accepts it, which can be done by a single reachability check on the product **DVPA** $\mathcal{A}_1 \times \mathcal{A}_2$, which simulates two copies of $\mathcal{A}$ with two stacks. However, the visibility condition ensures that two stacks can be simulated with one over the product of stack alphabets. Checking positive (resp., negative) consistency amounts to checking the existence of a word accepted by $\mathcal{A}_1$ but not $\mathcal{A}_2$ (resp., $\mathcal{A}_2$ but not $\mathcal{A}_1$ for negative consistency), which also can be done with a single reachability check.

▶ **Proposition 5.** *Consistency (resp., positive, negative consistency) of a ground well-matched rule with the language of a given **DVPA** $\mathcal{A}$ can be decided in polynomial time via a single reachability check on a **DVPA** obtained from the product of $\mathcal{A}$ with itself.*

Reachability in pushdown automata can be checked in $\mathcal{O}(|\mathcal{A}|^3)$ with a slightly better bound $\mathcal{O}(|\mathcal{A}|^3/\log(|\mathcal{A}|))$ for *recursive state machines* [19], which is still $\widetilde{\mathcal{O}}(|\mathcal{A}|^3)$. Due to the lack of better upper bounds [19, 8], the complexity of reachability in pushdown systems is often referred to as the "cubic bottleneck." Consequently, reachability in the product $\mathcal{A} \times \mathcal{A}$ amounts to $\widetilde{\mathcal{O}}(|\mathcal{A}|^6)$. In the case of **DFA**, however, checking consistency of an SRS

can be performed locally on the structure of the **DFA** without checking reachability. The following example illustrates that for pushdown systems, checking consistency must involve reachability.

▶ **Example 6.** Consider a **DVPA** $\mathcal{B}$ depicted in Figure 1, which is over the pushdown alphabet $(\{c\}, \{l_1, l_2\}, \{r\})$ such that local letters l_1, l_2 induce self-loops over all states except for q_1. Its stack alphabet is a singleton $\{a\}$. In the state q_1 the **DVPA** moves over l_1 to q_2 and over l_2 to q_2. The states q_2, q_3 have different behavior over the empty stack only, while they are indistinguishable over a non-empty stack. Finally, q_1 is reachable only with non-empty stack, i.e., q_1 is reachable, but the configuration $(q_1, \bot)$ is not. Then, the rewrite rule $l_1 \to l_2$ is consistent with $\mathcal{L}(\mathcal{B})$, but it requires reachability analysis for that.

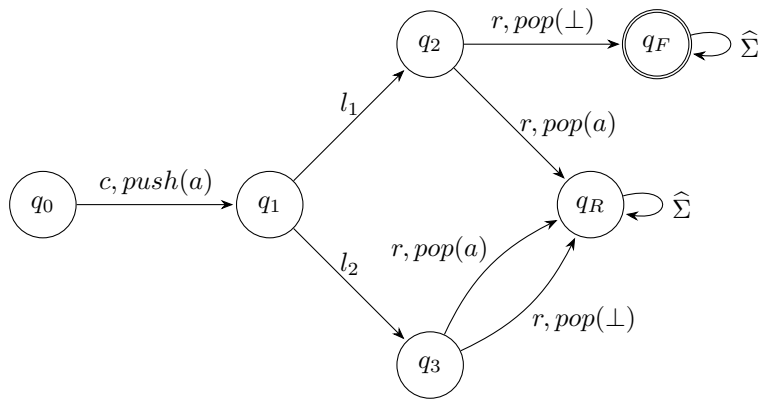


Figure 1 The automaton $\mathcal{B}$. All transitions other than depicted are self-loops

Non-ground well-matched rules. Consider a well-matched rule $\boldsymbol{\alpha} \to \boldsymbol{\beta}$ with variables. The general approach to decide consistency is, as in the ground case, to reduce it to reachability in a **VPA** $\mathcal{A}_{\text{con}}$. This automaton simulates two copies of $\mathcal{A}$ over the alphabet extended with a fresh local letter $\#$. The **VPA** $\mathcal{A}_{\text{con}}$ is based on the product $\mathcal{A} \times \mathcal{A}$, where the stack simulates the stacks of both copies of the **DVPA** $\mathcal{A}$. The fresh local letter $\#$ simulates the word $\boldsymbol{\alpha}$ in the first copy and the word $\boldsymbol{\beta}$ in the second copy. However, there are two main differences from the ground case. First, as $\boldsymbol{\alpha}$ and $\boldsymbol{\beta}$ can be instantiated to (infinitely many) different words, transitions over $\#$ are non-deterministic. Second, the transitions in each copy of $\mathcal{A}$ cannot be defined independently; they need to be define on the product to ensure that $\boldsymbol{\alpha}$ and $\boldsymbol{\beta}$ are instantiated to the same word. E.g., for a rule $cXr \to X$, there may be a transition in $\mathcal{A}$ from q to q_1 over cw_1r and from q to q_2 over w_2, but we must ensure that $w_1 = w_2$. Therefore, we compute a set $\Delta[\boldsymbol{\alpha} \to \boldsymbol{\beta}]$ triples (q, s_1, s_2) such that there are α, β satisfying: (1) $(q, u) \to^{\alpha}_{\mathcal{A}} (s_1, u)$ and $(q, u) \to^{\beta}_{\mathcal{A}} (s_2, u)$ for every stack content u, and (2) $\alpha \to \beta$ is an instance of $\boldsymbol{\alpha} \to \boldsymbol{\beta}$ obtained by substituting well-matched words for variables. Using $\Delta[\boldsymbol{\alpha} \to \boldsymbol{\beta}]$, we define a **VPA** $\mathcal{A}_{\text{con}}$ over the pushdown alphabet $\widehat{\Sigma}$ extended with a local letter $\#$ as an extension of $\mathcal{A} \times \mathcal{A}$ such that for a state (q_1, q_2) of $\mathcal{A}_{\text{con}}$: (a) if $q_1 \neq q_2$, then $\mathcal{A}_{\text{con}}$ has no transition over $\#$, and (b) if $q_1 = q_2$, then for every $(q_1, s_1, s_2) \in \Delta[\boldsymbol{\alpha} \to \boldsymbol{\beta}]$, the tuple $\langle (q_1, q_2), \#, (s_1, s_2) \rangle$ is a transition in $\mathcal{A}_{\text{con}}$. Finally, the automaton accepts if in the reached state (q_1, q_2), exactly one of q_1, q_2 is accepting in $\mathcal{A}$. Observe that the **VPA** $\mathcal{A}_{\text{con}}$ accepts exactly words w that are witnesses of inconsistency of $\boldsymbol{\alpha} \to \boldsymbol{\beta}$ with $\mathcal{L}(\mathcal{A})$. These are words w with exactly one occurrence of $\#$ such that there is an instance $\alpha \to \beta$ of $\boldsymbol{\alpha} \to \boldsymbol{\beta}$ obtained by substituting well-matched word for variables, and by substituting $\#$ with α and β we obtain words w_1, w_2 respectively satisfying (i) $w_1 \to w_2$, and (ii) exactly one of w_1, w_2

belongs to $\mathcal{L}(\mathcal{A})$. By modifying the acceptance condition, we can also decide positive and negative consistency.

In the following, we focus on computing $\Delta[\boldsymbol{\alpha} \to \boldsymbol{\beta}]$ for rules with variables. It suffices to compute how well-matched words transform pairs of states of the **DVPA**.

Computing $\Delta[\boldsymbol{\alpha} \to \boldsymbol{\beta}]$. We compute a quaternary relation $\mathcal{B}_{(4)} \subseteq Q^4$, where $(q_1, q_2, s_1, s_2) \in \mathcal{B}_{(4)}$ if and only if there is a well-matched word w such that $q_1 \to^w_{\mathcal{A}} s_1$ and $q_2 \to^w_{\mathcal{A}} s_2$. It is computed iteratively in a similar way to reachability in pushdown automata.

▶ **Lemma 7.** *Given a **DVPA** $\mathcal{A}$, the relation $\mathcal{B}_{(4)}$ can be computed in polynomial time in the size of $\mathcal{A}$.*

Now, we extend the alphabet $\widehat{\Sigma}$ to $\widehat{\Sigma}_+$ with a pair of fresh local letters x^L and x^R for every variable X from $\boldsymbol{\alpha} \to \boldsymbol{\beta}$. Next, we substitute each variable X with its corresponding local letters: x^L in $\boldsymbol{\alpha}$ and x^R in $\boldsymbol{\beta}$ and obtaining words α^+ and β^+ over the extended alphabet.

We compute $\mathcal{B}_{(4)}$ and then we iterate over all possible assignments from variables of $\boldsymbol{\alpha} \to \boldsymbol{\beta}$ to $\mathcal{B}_{(4)}$. For a given assignment σ, we extend the **DVPA** $\mathcal{A}$ to a **VPA** $\mathcal{A}_\sigma$ working over the extended alphabet. If $\sigma(X) = (q_1, q_2, q_1', q_2')$, we define the transitions for the new letters as follows: $\mathcal{A}_\sigma$ has a transition over x^L from q_1 to q_1' and over x^R from q_2 to q_2'; there are no other transitions over x^L or x^R. Finally, we evaluate $\boldsymbol{\alpha}$ and $\boldsymbol{\beta}$ by evaluating α^+ and β^+ in $\mathcal{A}_\sigma$. If $(q, \bot) \to^{\alpha^+}_{\mathcal{A}_\sigma} (s_1, \bot)$ and $(q, \bot) \to^{\beta^+}_{\mathcal{A}_\sigma} (s_2, \bot)$, we add the triple (q, s_1, s_2) to $\Delta[\boldsymbol{\alpha} \to \boldsymbol{\beta}]$. Not that the stack content $\bot$ does not influence reachability since α^+, β^+ are well-matched.

▶ **Lemma 8.** *Deciding membership to $\Delta[\boldsymbol{\alpha} \to \boldsymbol{\beta}]$ over well-matched rules is in NP. It can be done in polynomial time assuming a bound on the number of variables in rules.*

In consequence, checking if there is an inconsistency an be done in NPand hence:

▶ **Theorem 9.** *Consistency (resp., positive, negative consistency) of a well-matched rule with the language of a given **DVPA** $\mathcal{A}$ can be decided in coNP. Assuming a bound on the number of variables in rules, consistency (resp., positive, negative consistency) of a well-matched rule can be check in polynomial time.*

Complexity of checking (positive) consistency. The proposed algorithm checks all assignments of transitions for variables, which is exponential in the number of variables, but it works in polynomial time if the number of variables is bounded. Improving the proposed exponential-time algorithm is unlikely as the membership problem for $\Delta[\boldsymbol{\alpha} \to \boldsymbol{\beta}]$ is NP-complete in general. The hardness result hols already for **DFA**, which can be considered as **DVPA** over only local letters. Furthermore, the problem of deciding positive (resp. negative) consistency is coNP-complete as well.

▶ **Theorem 10.** *(1) The problem, given a rewrite rule $\boldsymbol{\alpha} \to \boldsymbol{\beta}$, a DFA $\mathcal{A}$ and its states q, s_1, s_2, does the tuple (q, s_1, s_2) belongs to $\Delta[\boldsymbol{\alpha} \to \boldsymbol{\beta}]$, is NP-complete. (2) The positive (resp. negative) consistency problem over well-matched rules against a regular language given by a **DFA** is coNP-complete.*

The above coNP-hardness result does not extend to consistency and hence the complexity lower bound for checking consistency of well-matched rules remains an open problem.

Now, we discuss matching-preserving rules, which are not well-matched. The overall approach is similar to the well-matched; we extend the alphabet with auxiliary letters. In contrast to well-matched rules, the new letters are call and return letters.

Ground matching-preserving rules. Consider a matching preserving rule $\boldsymbol{\alpha} \to \boldsymbol{\beta}$ without variables, which is not a well-matched rule. Observe that $\boldsymbol{\alpha}$ and $\boldsymbol{\beta}$ have the same number of unmatched calls m and unmatched returns k, and since it is not well-matched, there are unmatched letters (at least one of k, m is non-zero). Furthermore, $\boldsymbol{\alpha}$ and $\boldsymbol{\beta}$ can be partitioned into well-matched words and unmatched calls and returns, where unmatched returns precede unmatched calls. It follows that

$$\boldsymbol{\alpha} = v_0 r_1 v_1 \dots r_k v_k c_1 v_{k+1} c_2 \dots c_m v_{k+m}$$
$$\boldsymbol{\beta} = v'_0 r'_1 v'_1 \dots r'_k v'_k c'_1 v'_{k+1} c'_2 \dots c'_m v'_{k+m}$$

where v_i, v'_i are well-matched word, r_i, r'_i are return letters and c_i, c'_i are call letters. Now, as in the well-matched case, consider a **DVPA** $\mathcal{A}_E$ being the product of $\mathcal{A}$ with itself, i.e., the set of states is $Q \times Q$ and the stack alphabet is $\Gamma \times \Gamma$. The automaton $\mathcal{A}_E$ works over an extended alphabet, where the additional letters are k fresh return letters $\bar{r}_1, \dots, \bar{r}_k$ and m fresh call letters $\bar{c}_1, \dots, \bar{c}_m$. The transitions over the letter $\bar{r}_1$ simulates the word $v_0 r_1 v_1$ on the first component of $\mathcal{A}_E$ and it simulates the word $v'_0 r'_1 v'_1$ on the second component. While doing so, it pops a single letter from the stack, which is a pair of stack symbols of $\mathcal{A}$. Formally, $\delta_E((q_1, q_2), \bar{r}_i, (b_1, b_2)) = (s_1, s_2)$ if $(q_1, ub_1) \to_{\mathcal{A}}^{v_0 r_1 v_1} (s_1, u)$ and $(q_2, ub_2) \to_{\mathcal{A}}^{v'_0 r'_1 v'_1} (s_2, u)$ for every stack content u. For the remaining return letters $\bar{r}_i$, where $i \in \{2, \dots, k\}$ the transitions on $\bar{r}_i$ are equivalent to processing $r_i v_i$ on the first component and $r'_i v'_i$ on the second component. For new call letter $\bar{c}_i$ the transition over $\bar{c}_i$ is equivalent to the transition over the word $c_i v_{k+1}$ on the first component and $c'_i v'_{i+k}$ on the second component. Finally, a state (q_1, q_2) of $\mathcal{A}_E$ are accepting if exactly one of q_1, q_2 is accepting in $\mathcal{A}$.

Observe that a word $w_1 \bar{r}_1 \dots \bar{r}_k \bar{c}_1 \cdots \bar{c}_m w_2$, where w_1, w_2 are over the original alphabet $\widehat{\Sigma}$, is accepted by $\mathcal{A}_E$ if and only if $\mathcal{A}$ accepts exactly one of the words $w_1 \boldsymbol{\alpha} w_2, w_1 \boldsymbol{\beta} w_2$. The latter is equivalent to $\boldsymbol{\alpha} \to \boldsymbol{\beta}$ being not consistent with $\mathcal{L}(\mathcal{A})$. Therefore, the rule $\boldsymbol{\alpha} \to \boldsymbol{\beta}$ is consistent with $\mathcal{L}(\mathcal{A})$ if and only if the language $\mathcal{L}(\mathcal{A}_E) \cap \mathcal{L}(\widehat{\Sigma}^* \bar{r}_1 \dots \bar{r}_k \bar{c}_1 \cdots \bar{c}_m \widehat{\Sigma}^*)$ is empty.

By changing the accepting states of $\mathcal{A}_E$ as discussed in the well-matched case, we can decide positive and negative consistency using emptiness of $\mathcal{L}(\mathcal{A}_E) \cap \mathcal{L}(\widehat{\Sigma}^* \bar{r}_1 \dots \bar{r}_k \bar{c}_1 \cdots \bar{c}_m \widehat{\Sigma}^*)$. In consequence, we have:

▶ **Proposition 11.** *Consistency (resp., positive, negative consistency) of a ground matching-preserving rule with the language of a given **DVPA** $\mathcal{A}$ can be decided in polynomial time via a single reachability check on a **DVPA** obtained from the product of $\mathcal{A}$ with itself and a regular expression.*

Matching-preserving rules with variables. Variables in matching-preserving rules can be eliminated as in the case of well-matched rules. Consider a matching-preserving rule $\boldsymbol{\alpha} \to \boldsymbol{\beta}$. It suffices to extend the alphabet with local letters x^L, x^R for each variable and then consider a ground rule obtained from $\boldsymbol{\alpha} \to \boldsymbol{\beta}$ by replacing the occurrence of every variable X in $\boldsymbol{\alpha}$ with x^L and in $\boldsymbol{\beta}$ (if it exists) with x^R. Finally, we as in the well-matched case, we iterate over all possible assignments from variables of $\boldsymbol{\alpha} \to \boldsymbol{\beta}$ to $\mathcal{B}_{(4)}$. In consequence, we have

▶ **Theorem 12.** *Consistency (resp., positive, negative consistency) of a matching-preserving rule with the language of a given **DVPA** $\mathcal{A}$ can be decided in coNP. Assuming a bound on the number of variables in rules, consistency (resp., positive, negative consistency) of a matching-preserving rule can be check in polynomial time.*

4 Rule synthesis

The already studied problem is how advice provided via an SRS reduces the number of equivalence queries necessary to learn an unknown language. In this section addresses the converse: *synthesis* of rewrite rules consistent with a given visibly pushdown language. Synthesized rewrite rules can serve as an explanation of the considered visibly pushdown language, capturing its key properties through concise rewrite rules.

There are two possible variants of that problem:

Synthesis from Automata: Given a **DVPA**, generate string rewrite rules consistent with the language of the given **DVPA**.

Synthesis from Oracles: Given an oracle for an unknown visibly pushdown language, generate string rewrite rules consistent with the language.

While (*Synthesis from Oracles*) seems more natural, we show that even having a **DVPA** the synthesis problem is challenging for two reasons: high computational complexity and an undetermined objective (what is the right notion of a "relevant rule"?). Therefore, (*Synthesis from Oracles*) suffers from these same shortcomings, and further progress in (*Synthesis from Automata*) is required before (*Synthesis from Oracles*) can be effectively addressed.

We consider a **DVPA** $\mathcal{A}$ over a non-trivial alphabet (at least two letters). Furthermore, we assume that if the alphabet includes a call letter, it also includes a return letter, and vice versa. The latter assumption is to ensure that there are other well-matched words than the empty word.

We establish a polynomial upper bound on the size of minimal-length rules consistent with $\mathcal{A}$, which implies that such rules can be computed in non-deterministic polynomial time.

mention that the bounds are not obvious?

In order to achieve that, we give necessary conditions for pairs of words to form a rewrite rule consistent with $\mathcal{L}(\mathcal{A})$. We consider first rules without variables, and then move to rules with a single variable, which is finally generalized to any bounded number of variables.

The size of a minimal consistent well-matched rule: the ground case. Consider a well-matched word w. We associate with this word a function $f_w \colon Q \to Q$ transforming states into states as the congruence relation for regular languages. Formally, for a state $q \in Q$ we define $f_w(q) = q'$ if and only if for some stack content u, we have $(q, u) \to^a_{\mathcal{A}} (q', u)$. Note that "some u" can be changed to "all u" as the transitions over well-matched word do not depend on the initial stack content. If w_1, w_2 are well-matched word such that $f_{w_1} = f_{w_2}$, then for all words x, y we have $xw_1y \in \mathcal{L}(\mathcal{A}) \iff xw_2y \in \mathcal{L}(\mathcal{A})$ [2, Theorem 1]. It follows that $w_1 \to w_2$ is a string rewrite rule consistent with $\mathcal{L}(\mathcal{A})$. Setting $n = |Q|$, there are n^n functions from Q to Q, and hence there are $2^{\mathcal{O}(n \log n)}$ possible functions f_w, where w is a well-matched words. There are $2^{\Theta}(k)$ well-matched words of the length k, and hence over well balanced words of length $\mathcal{O}(n \log n)$ there is a pair of a well-matched words $\boldsymbol{\alpha}, \boldsymbol{\beta}$ such that $\boldsymbol{\alpha} \to \boldsymbol{\beta}$ is a ground well-matched rule consistent with $\mathcal{L}(\mathcal{A})$.

The size of a minimal consistent well-matched rule: the general case. For rules with variables, we consider only a subset of rules and only a sufficient condition for consistency. Still, we obtain asymptotically the same bound. Consider a rewrite rule with variables $\boldsymbol{\alpha} \to \boldsymbol{\beta}$ of the form:

$$\boldsymbol{\alpha} = w_0 X_1 w_1 X_2 \dots X_k w_k$$
$$\boldsymbol{\beta} = v_0 X_1 v_1 X_2 \dots X_k v_k$$

where the order of variables in $\boldsymbol{\alpha}$ and $\boldsymbol{\beta}$ is the same and all words w_i, v_i all well-matched

words over $\widehat{\Sigma}$ (do not contain variables). Observe that if the corresponding words w_i and v_i define the same function, i.e., $f_{w_0} = f_{v_0}, \ldots, f_{w_k} = f_{v_k}$, then the rewrite rule $\boldsymbol{\alpha} \to \boldsymbol{\beta}$ is consistent with the language of $\mathcal{A}$. To see that, consider an instantiation of $\boldsymbol{\alpha} \to \boldsymbol{\beta}$, i.e., consider a sequence of balanced word $b_1, \ldots, b_k$, which we substitute for $X_1, \ldots, X_k$ obtaining the ground rewrite rule:

$$w_0 b_1 w_1 b_2 \ldots b_k w_k \to v_0 b_1 v_1 b_2 \ldots b_k v_k$$

Observe that the left hand side and the right hand side define the same function, and hence this ground rule is consistent with $\mathcal{L}(\mathcal{A})$. It follows that every rule from $[\![\boldsymbol{\alpha} \to \boldsymbol{\beta}]\!]$ is consistent with $\mathcal{L}(\mathcal{A})$, which by definition means that $\boldsymbol{\alpha} \to \boldsymbol{\beta}$ is consistent with $\mathcal{L}(\mathcal{A})$. Now, observe that, setting $n = |Q|$, there are $n^{k \cdot n}$ different combinations of k functions from Q to Q. The number of well-balanced words is $2^{\Omega(n)}$ and hence there is a constant $c > 0$ such that among words of the form $w_0 X_1 w_1 X_2 \ldots X_k w_k$ such that the sum of lengths of $w_0, \ldots, w_k$ bounded by $2^{c \cdot k \cdot n \cdot \log(n)}$, there is a pair of words, which forms a rewriting rule consistent with $\mathcal{L}(\mathcal{A})$.

The counting argument shows that for a given **DVPA** $\mathcal{A}$ there are short (of polynomial length in $|\mathcal{A}|$) rewrite rules that are consistent with $\mathcal{L}(\mathcal{A})$. Since testing whether a given rule (with bounded number of variables) is consistent with $\mathcal{L}(\mathcal{A})$ is decidable in polynomial time, we can find (some) rules consistent with $\mathcal{L}(\mathcal{A})$ in polynomial time. Clearly, not all rules can be generated in non-deterministic polynomial time. Even for the **DFA** case, we can construct an automaton $\mathcal{A}$ such that there are words w such that $w \to \epsilon$ is a rule consistent with $\mathcal{L}(\mathcal{A})$, but every such w is of exponential length. It suffices to define a DFA such that transitions over each letter is the same permutation of an exponential order.

While we have shown that rules can be generated in non-deterministic polynomial time, a natural question is whether this is optimal.

Complexity of finding consistent rules. We have established an upper bound on the minimal length of words defining the same function. We show that computing the exact minimal length of such words is difficult, which implies that finding a minimal-length rewriting rule, which is consistent with a given **DVPA** is difficult as well. Furthermore, the problem is difficult already for **DFA**.

We consider the *minimal collision* problem defined as follows: decide, given a **DFA** $\mathcal{A}$ and $k > 0$, whether there exists a pair of different words w_1, w_2 such that (1) $|w_1|, |w_2| \leq k$, and (2) $f_{w_1} = f_{w_2}$. As we have discussed above for the ground rewrite rules, assuming that $\mathcal{A}$ is over at least two-letter alphabet, for every $k > |\mathcal{A}| \cdot \log(|\mathcal{A}|)$ the answer is yes. Consequently, the problem is in NP. We show that it is NP-complete.

▶ **Lemma 13.** *The minimal collision problem is NP-complete.*

The proof is by reduction from the 3-SAT problem. The main difficulty of the proof is to ensure that if the formula is unsatisfiable, then no pair of short words define the same function from Q to Q . We have constructed a gadget **DFA**, such that unless one of the designed letters occurs in a word, each word of length at most n generates the unique function. We consider the alphabet consisting of literals from a given 3-CNF formula. With that gadget **DFA**, we enforce that short words having a collision, must contain a variable or its negation, i.e., represent an assignment, and these word must contain a literal from each clause.

Another decision problem associated with finding consistent rules is deciding, for a given **DFA** $\mathcal{A}$ and $v \in \Sigma^*$, the minimal length of w such that $f_w = f_v$. We formalize this problem as follows. The *bounded rewriting to single letter* problem asks, given a **DFA** $\mathcal{A}$ over Σ, a letter $a \in \Sigma$, and a bound $k > 0$, whether there exists a word $w \in (\Sigma \setminus \{a\})^*$ such that (1) $w \to a$ is a rule consistent with $\mathcal{L}(\mathcal{A})$, and (2) $|w| \leq k$. This problem is closely

related to the *membership* problem for monoids [37] and groups [33], which in turn reduce to the *bounded automata intersection* problem [37, 54], which asks, given $m, k > 0$ and m **DFA** $\mathcal{A}_1, \ldots, \mathcal{A}_m$, whether there is w such that $|w| \leq k$ and $w \in n\mathcal{L}(\mathcal{A}_i)$ for every i. The bounded automata intersection problem is PSPACE-complete if k is given in binary [37] and NP-complete if k is given in unary [54]. The same complexity bounds apply to the bounded rewriting to single letter problem as there is a straightforward reduction from the bounded automata intersection problem:

▶ **Lemma 14.** *The bounded automata intersection problem with length parameter k reduces in polynomial time to bounded rewriting to a single letter with the parameter k.*

In summary, there is not algorithm for generating minimal-length consistent rules in deterministic polynomial time unless P= NP. Consequently, the further research on the rule synthesis problem should focus on efficient heuristics rather than exact algorithms.

5 Empirical evaluation

We evaluate our framework by expressing natural VPL properties as rewrite rules with variables. Our experiments confirm that this learning with advice approach yields a substantial reduction in equivalence queries, confirming the practical advantages of the proposed method.

5.1 Examples of rewrite rules

We consider the following types rules:

- *Well-matched commutativity*: $c_1c_2Xr_2r_1 \to c_2c_1Xr_1r_2$.
- *Well-matched join*: $cXrcYr \to cXYr$.
- *Well-matched cancellation*: $cXr \to \epsilon$.

  c, r are comment tags
- *Well-matched idempotency*: $ccXrr \to cXr$.

5.2 Implementation

Baseline. As a baseline for our experiments, we have implemented a polynomial-time active learning algorithm for **DVPA** [44]. The algorithm from [44] assumes a stronger teacher model than the typical MAT; it allows queries extended membership queries that read as well modify the stack content and hence it depends on a concrete **DVPA** rather than **VPL**. On the positive side, the learning algorithm works in polynomial time.

The advice mechanism. We have extended the baseline algorithm with the advice mechanism, where the advice is given via rewrite rules with a **single** variable.

how the advice is computed

Equivalence tests.

two types of equivalence checks – exact and probabilistic

5.3 Experiments

Reserch questions:

Benchmarks.

Note that the number of equivalence queries depends on the number of states, not the size of the alphabet

Scenarios.

Perefect eqivalence and probabilistic

We preset the results per rule.

Results on well-matched cancellation. $cr \to \epsilon$ usually does the trick

Results on well-matched commutativity. Idempotency

Results on well-matched idempotency.

Summary.

References

1. Andreas Abel and Jan Reineke. Gray-box learning of serial compositions of mealy machines. In *NFM 2016*, pages 272–287, 2016.
2. Rajeev Alur, Viraj Kumar, Parthasarathy Madhusudan, and Mahesh Viswanathan. Congruences for visibly pushdown languages. In *ICALP 2005*, pages 1102–1114. Springer, 2005.
3. Rajeev Alur and P. Madhusudan. Visibly pushdown languages. In László Babai, editor, *Proceedings of the 36th Annual ACM Symposium on Theory of Computing, Chicago, IL, USA, June 13-16, 2004*, pages 202–211. ACM, 2004.
4. Rajeev Alur and P. Madhusudan. Adding nesting structure to words. *J. ACM*, 56(3):16:1–16:43, 2009.
5. Dana Angluin. Learning k-bounded context-free grammars. *Research report. Yale University. Dept. of Computer Science*, 557, 1987.
6. Dana Angluin. Learning regular sets from queries and counterexamples. *Information and computation*, 75(2):87–106, 1987.
7. Dana Angluin and Michael Kharitonov. When won't membership queries help? *Journal of Computer and System Sciences*, 50(2):336–355, 1995.
8. A. R. Balasubramanian, Dmitry Chistikov, and Rupak Majumdar. Pushdown model checking above the cubic bottleneck. In *40th Annual ACM/IEEE Symposium on Logic in Computer Science, LICS 2025, Singapore, June 23-26, 2025*, pages 622–635. IEEE, 2025.
9. Benoît Barbot, Benedikt Bollig, Alain Finkel, Serge Haddad, Igor Khmelnitsky, Martin Leucker, Daniel Neider, Rajarshi Roy, and Lina Ye. Extracting context-free grammars from recurrent neural networks using tree-automata learning and a* search. In Jane Chandlee, Rémi Eyraud, Jeff Heinz, Adam Jardine, and Menno van Zaanen, editors, *Proceedings of the 15th International Conference on Grammatical Inference, 23-27 August 2021, Virtual Event*, Proceedings of Machine Learning Research, pages 113–129. PMLR, 2021.
10. Osbert Bastani, Rahul Sharma, Alex Aiken, and Percy Liang. Synthesizing program input grammars. In Albert Cohen and Martin T. Vechev, editors, *Proceedings of the 38th ACM SIGPLAN Conference on Programming Language Design and Implementation, PLDI 2017, Barcelona, Spain, June 18-23, 2017*, pages 95–110. ACM, 2017.
11. Amos Beimel, Francesco Bergadano, Nader Bshouty, Eyal Kushilevitz, and Stefano Varricchio. Learning functions represented as multiplicity automata. *Journal of the ACM*, 47, 10 1999.
12. Bachir Bendrissou, Rahul Gopinath, and Andreas Zeller. "synthesizing input grammars": a replication study. In Ranjit Jhala and Isil Dillig, editors, *PLDI '22: 43rd ACM SIGPLAN International Conference on Programming Language Design and Implementation, San Diego, CA, USA, June 13 - 17, 2022*, pages 260–268. ACM, 2022.
13. Raphaël Berthon, Adrien Boiret, Guillermo A. Pérez, and Jean-François Raskin. Active learning of sequential transducers with side information about the domain. In *DLT 2021*, pages 54–65, 2021.
14. Satwik Bhattamishra, Michael Hahn, and Varun Kanade. Automata learning and identification of the support of language models. In *International Conference on Learning Representations ICLR 2026*. OpenReview.net, 2026.

15 Henrik Björklund, Johanna Björklund, and Petter Ericson. On the regularity and learnability of ordered DAG languages. In Arnaud Carayol and Cyril Nicaud, editors, *CIAA 2017*, volume 10329 of *Lecture Notes in Computer Science*, pages 27–39. Springer, 2017.

16 Benedikt Bollig, Peter Habermehl, Carsten Kern, and Martin Leucker. Angluin-style learning of NFA. In *IJCAI 2009*, pages 1004–1009, 2009.

17 Ronald V. Book and Friedrich Otto. *String-Rewriting Systems*. Texts and Monographs in Computer Science. Springer, 1993.

18 Véronique Bruyère, Guillermo A. Pérez, and Gaëtan Staquet. Validating streaming JSON documents with learned vpas. In Sriram Sankaranarayanan and Natasha Sharygina, editors, *TACAS 2023*, Lecture Notes in Computer Science, pages 271–289. Springer, 2023.

19 Swarat Chaudhuri. Subcubic algorithms for recursive state machines. In George C. Necula and Philip Wadler, editors, *Proceedings of the 35th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages, POPL 2008, San Francisco, California, USA, January 7-12, 2008*, pages 159–169. ACM, 2008.

20 Patrick Chervet and Igor Walukiewicz. Minimizing variants of visibly pushdown automata. In Ludek Kucera and Antonín Kucera, editors, *Mathematical Foundations of Computer Science 2007, 32nd International Symposium, MFCS 2007, Ceský Krumlov, Czech Republic, August 26-31, 2007, Proceedings*, Lecture Notes in Computer Science, pages 135–146. Springer, 2007.

21 Colin de la Higuera. *Grammatical Inference: Learning Automata and Grammars*. Cambridge University Press, New York, NY, USA, 2010.

22 Simon Dierl, Paul Fiterau-Brostean, Falk Howar, Bengt Jonsson, Konstantinos Sagonas, and Fredrik Tåquist. Scalable tree-based register automata learning. In *TACAS 2024*, pages 87–108, 2024.

23 Frank Drewes and Johanna Högberg. Learning a regular tree language from a teacher. In Zoltán Ésik and Zoltán Fülöp, editors, *DLT 2003*, volume 2710 of *Lecture Notes in Computer Science*, pages 279–291. Springer, 2003.

24 Kévin Dubrulle, Véronique Bruyère, Guillermo A. Pérez, and Gaëtan Staquet. Visibly recursive automata. *CoRR*, abs/2603.11648, 2026.

25 Karim Elmaaroufi, Marcell Vazquez-Chanlatte, Stefan Witwicki, Matei Zaharia, and Sanjit A. Seshia. Learning automata from demonstrations, examples, and natural language. In *ICLR 2025 Workshop on Building Trust in Language Models and Applications*, 2025.

26 Michal Fica and Jan Otop. Active automata learning with advice. In Inês Lynce, Nello Murano, Mauro Vallati, Serena Villata, Federico Chesani, Michela Milano, Andrea Omicini, and Mehdi Dastani, editors, *ECAI 2025 - 28th European Conference on Artificial Intelligence, 25-30 October 2025, Bologna, Italy - Including 14th Conference on Prestigious Applications of Intelligent Systems (PAIS 2025)*, Frontiers in Artificial Intelligence and Applications, pages 1655–1662. IOS Press, 2025.

27 Markus Theo Frohme. Active automata learning with adaptive distinguishing sequences. *CoRR*, abs/1902.01139, 2019.

28 Olivier Gauwin, Anca Muscholl, and Michael Raskin. Minimization of visibly pushdown automata is np-complete. *Log. Methods Comput. Sci.*, 16(1), 2020.

29 John Hewitt, Michael Hahn, Surya Ganguli, Percy Liang, and Christopher D. Manning. Rnns can generate bounded hierarchical languages with optimal memory. In Bonnie Webber, Trevor Cohn, Yulan He, and Yang Liu, editors, *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing, EMNLP 2020, Online, November 16-20, 2020*, pages 1978–2010. Association for Computational Linguistics, 2020.

30 Malte Isberner. *Foundations of active automata learning: an algorithmic perspective*. PhD thesis, Technical University Dortmund, Germany, 2015.

31 Malte Isberner, Falk Howar, and Bernhard Steffen. The TTT algorithm: A redundancy-free approach to active automata learning. In *RV 2014*, pages 307–322, 2014.

32 Petr Jancar. Equivalence of pushdown automata via first-order grammars. *J. Comput. Syst. Sci.*, 115:86–112, 2021.

33 Mark Jerrum. The complexity of finding minimum-length generator sequences. *Theor. Comput. Sci.*, 36:265–289, 1985.

34 Xiaodong Jia and Gang Tan. V-star: Learning visibly pushdown grammars from program inputs. *Proc. ACM Program. Lang.*, 8(PLDI):2003–2026, 2024.

35 Bruce Knobe and Kathleen Knobe. A method for inferring context-free grammars. *Information and Control*, 31(2):129–146, 1976.

36 Rick Koenders and Joshua Moerman. Output-decomposed learning of mealy machines. *CoRR*, abs/2405.08647, 2024.

37 Dexter Kozen. Lower bounds for natural proof systems. In *18th Annual Symposium on Foundations of Computer Science, Providence, Rhode Island, USA, 31 October - 1 November 1977*, pages 254–266. IEEE Computer Society, 1977.

38 Loes Kruger, Sebastian Junges, and Jurriaan Rot. Small test suites for active automata learning. In *TACAS 2024*, pages 109–129, 2024.

39 Neil Kulkarni, Caroline Lemieux, and Koushik Sen. Learning highly recursive input grammars. In *36th IEEE/ACM International Conference on Automated Software Engineering, ASE 2021, Melbourne, Australia, November 15-19, 2021*, pages 456–467. IEEE, 2021.

40 Viraj Kumar, P. Madhusudan, and Mahesh Viswanathan. Minimization, learning, and conformance testing of boolean programs. In Christel Baier and Holger Hermanns, editors, *CONCUR 2006 - Concurrency Theory, 17th International Conference, CONCUR 2006, Bonn, Germany, August 27-30, 2006, Proceedings*, Lecture Notes in Computer Science, pages 203–217. Springer, 2006.

41 Faezeh Labbaf, Jan Friso Groote, Hossein Hojjat, and Mohammad Reza Mousavi. Compositional learning for interleaving parallel automata. In Orna Kupferman and Pawel Sobocinski, editors, *FoSSaCS 2023*, volume 13992 of *Lecture Notes in Computer Science*, pages 413–435. Springer, 2023.

42 Matteo Magnini, Riccardo Squarcialupi, Martin Tunge Sterri, Ana Ozaki, and Rivera Castillo. Actively learning el terminologies from large language models. In *ECAI 2025*, volume 413 of *Frontiers in Artificial Intelligence and Applications*, pages 1792–1799. IOS Press, 2025.

43 Kurt Mehlhorn. Pebbling moutain ranges and its application of dcfl-recognition. In J. W. de Bakker and Jan van Leeuwen, editors, *Automata, Languages and Programming, 7th Colloquium, Noordweijkerhout, The Netherlands, July 14-18, 1980, Proceedings*, Lecture Notes in Computer Science, pages 422–435. Springer, 1980.

44 Jakub Michaliszyn and Jan Otop. Learning deterministic visibly pushdown automata under accessible stack. In Stefan Szeider, Robert Ganian, and Alexandra Silva, editors, *47th International Symposium on Mathematical Foundations of Computer Science, MFCS 2022, Vienna, Austria, August 22-26, 2022*, LIPIcs, pages 74:1–74:16. Schloss Dagstuhl - Leibniz-Zentrum für Informatik, 2022.

45 Joshua Moerman. Learning product automata. In Olgierd Unold, Witold Dyrka, and Wojciech Wieczorek, editors, *ICGI 2018*, volume 93 of *Proceedings of Machine Learning Research*, pages 54–66. PMLR, 2018.

46 Joshua Moerman, Matteo Sammartino, Alexandra Silva, Bartek Klin, and Michal Szynwelski. Learning nominal automata. In *POPL 2017*, pages 613–625, 2017.

47 Yasubumi Sakakibara. Efficient learning of context-free grammars from positive structural examples. *Information and Computation*, 97(1):23–60, 1992.

48 Géraud Sénizergues. The equivalence problem for deterministic pushdown automata is decidable. In *ICALP 1997*, pages 671–681, 1997.

49 Luzi Sennhauser and Robert C. Berwick. Evaluating the ability of LSTMs to learn context-free grammars. In *Proceedings of the Workshop: Analyzing and Interpreting Neural Networks for NLP, BlackboxNLP@EMNLP 2018*, pages 115–124, 2018.

50 Ehud Y Shapiro. Algorithmic program diagnosis. In *Proceedings of the 9th ACM SIGPLAN-SIGACT symposium on Principles of programming languages*, pages 299–308, 1982.

51 Frits Vaandrager. Model learning. *Communications of the ACM*, 60(2):86–95, 2017.

52 Frits W. Vaandrager, Bharat Garhewal, Jurriaan Rot, and Thorsten Wißmann. A new approach for active automata learning based on apartness. In *TACAS 2022*, pages 223–243, 2022.

53 Gerco van Heerdt, Tobias Kappé, Jurriaan Rot, and Alexandra Silva. Learning pomset automata. In Stefan Kiefer and Christine Tasson, editors, *ETAPS 2021*, volume 12650, pages 510–530. Springer, 2021.

54 Todd Wareham. The parameterized complexity of intersection and composition operations on sets of finite-state automata. In Sheng Yu and Andrei Paun, editors, *Implementation and Application of Automata, 5th International Conference, CIAA 2000, London, Ontario, Canada, July 24-25, 2000, Revised Papers*, Lecture Notes in Computer Science, pages 302–310. Springer, 2000.

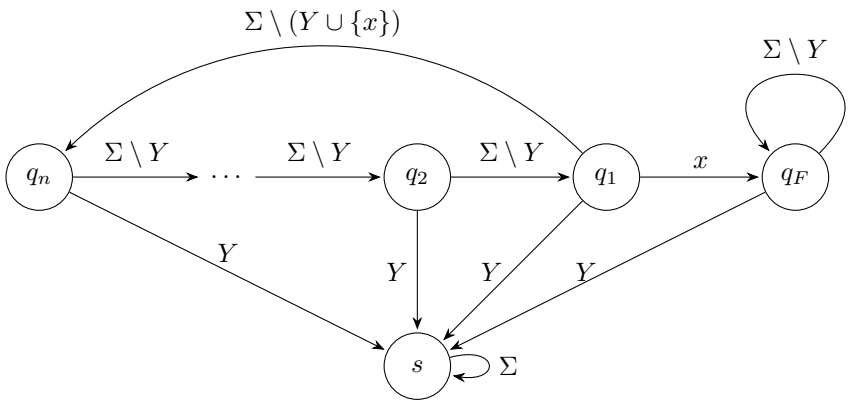
55 Zhengkai Wu, Evan Johnson, Wei Yang, Osbert Bastani, Dawn Song, Jian Peng, and Tao Xie. REINAM: reinforcement learning for input-grammar inference. In Marlon Dumas, Dietmar Pfahl, Sven Apel, and Alessandra Russo, editors, *Proceedings of the ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering, ESEC/SIGSOFT FSE 2019, Tallinn, Estonia, August 26-30, 2019*, pages 488–498. ACM, 2019.

▶ **Theorem 10.** *(1) The problem, given a rewrite rule $\boldsymbol{\alpha} \to \boldsymbol{\beta}$, a DFA $\mathcal{A}$ and its states q, s_1, s_2, does the tuple (q, s_1, s_2) belongs to $\Delta[\boldsymbol{\alpha} \to \boldsymbol{\beta}]$, is NP-complete. (2) The positive (resp. negative) consistency problem over well-matched rules against a regular language given by a **DFA** is coNP-complete.*

Proof. ◀

▶ **Lemma 15.** *The minimal collision problem is NP-complete.*

Proof. Let us fix a parameter n. Before the reduction, we construct a gadget that distinguishes all words of length at most n unless they consist one of the designated letters. Let $Y \subseteq \Sigma$ and $x \in \Sigma \setminus Y$. Consider the automaton $\mathcal{B}_{Y,x}$ depicted below:



Observe that for all words w_1, w_2 of length bounded by n, if $f_{w_1}^{\mathcal{B}_{Y,x}} = f_{w_2}^{\mathcal{B}_{Y,x}}$, then one of the following holds:

- each of the words w_1, w_2 contains at least one letter from Y; in that case f_{w_1}, f_{w_2} are constant functions equal s for every argument,
- the sets positions at which x occurs in w_1 and resp. w_2 are equal; indeed, for any $w \in (\Sigma \setminus Y)^{\leq n}$ we have $f_w(q_i) = q_F$ if and only if $w[i] = x$.

Now, consider $\mathcal{B}_Y$ being the disjoint union of automata $\mathcal{B}_{Y,x}$ over all $x \in \Sigma \setminus Y$.

Observe that for all words w_1, w_2 of length bounded by n, if $f_{w_1}^{\mathcal{B}_Y} = f_{w_2}^{\mathcal{B}_Y}$, then one of the following holds:

- each of the words w_1, w_2 contains at least one letter from Y; in that case f_{w_1}, f_{w_2} are constant functions equal s for every argument,
- words w_1 and w_2 are equal.

Having the automaton $\mathcal{B}_Y$ we reduce the 3-SAT problem to the minimal collision problem. Let φ be a 3-CNF formula over variables $x_1, \ldots, x_n$. Consider $\Sigma = \{x_1, \bar{x_1}, \ldots, x_n, \bar{x_n}, a, b\}$. With each clause c_j we associate the subset C_j of Σ consisting of the literals from c_j. Furthermore, for every variable x_i we define the set $X_i = \{x_i, \bar{x_i}\}$. Now, consider a **DFA** $\mathcal{A}_0$ being the disjoint union of automata $\mathcal{B}_{C_1}, \ldots, \mathcal{B}_{C_m}$ over all clauses of φ and automata $\mathcal{B}_{X_1}, \ldots, \mathcal{B}_{X_n}$ over all variables of φ. We extend $\mathcal{A}_0$ to $\mathcal{A}$ with additional states such that over these states letters from $\Sigma \setminus \{a, b\}$ are self loops, and letters a, b ensure that every component of the disjoint union is reachable. Let k in the instance of the minimal collision problem be equal n, the number of variables.

Now, consider a pair of different words w_1, w_2 such that $f^{\mathcal{A}}_{w_1} = f^{\mathcal{A}}_{w_2}$ and $|w_1|, |w_2| \leq n$. Then, for every automaton $\mathcal{B}_{X_1}, \ldots, \mathcal{B}_{X_n}$, since $w_1 \neq w_2$, each word contains at least one letter from each set X_i. There are n such sets, which are disjoint, and hence w_1 and w_2 contain exactly one letter from each of the sets X_i, i.e., w_1 and w_2 represent consistent variable assignments (they either contain x_i or $\bar{x_i}$). Furthermore, for every clause c_j each word w_1, w_2 contains at least one letter corresponding to some literal from c_j. Therefore, w_1 and w_2 represent (possibly different) satisfying assignments for φ.

Conversely, assume that φ has a satisfying assignment. Consider two words w_1, w_2 representing this variable assignment with a different order or literals. Then, for every component $\mathcal{B}_{Y,x}$ of $\mathcal{A}_0$, the functions $f^{\mathcal{B}_{Y,x}}_{w_1}, f^{\mathcal{B}_{Y,x}}_{w_2}$ are constant functions equal s. Furthermore, on states in $\mathcal{A}$, which are not in $\mathcal{A}_0$, both functions $f^{\mathcal{A}}_{w_1}, f^{\mathcal{A}}_{w_2}$ coincide with the identity. Consequently, we have (a) $w_1 \neq w_2$, (b) $|w_1| = |w_2| = n$, and (c) $f^{\mathcal{A}_0}_{w_1}, f^{\mathcal{A}_0}_{w_2}$, i.e., the constructed instance of the minimal collision problem is positive. ◀